OPENEUROLLM · TECHNICAL BLUEPRINT · 2026

The 2026 Fully Open LLM Training Guide

A comprehensive, citation-grounded, systems-level guide for training, evaluating, post-training, and releasing a fully open multilingual foundation model.

Data lineage | Dense + MoE architecture | GRPO / RLHF / DPO | Agent training | Open release

Date
2026-05-07

Format
Academic technical report

Scope
Fully open LLM lifecycle

Contents

Abstract

This document is a practical, academic-style guide to training a fully open large language model in 2026. It is grounded in the OpenEuroLLM public mission: transparent, compliant, open-source multilingual foundation models for Europe and beyond, with open documentation, training and testing code, evaluation metrics, intermediate results, and community involvement [@openeurollm_official_2026; @ai_sweden_openeurollm_2026]. This public edition uses OpenEuroLLM as an illustrative target for open, multilingual, European foundation-model engineering and avoids non-public project status claims.

The guide covers the full lifecycle: project governance, data acquisition, curation, multilingual mixture design, tokenizer training, architecture selection, scaling-law experiments, HPC infrastructure, pretraining, mid-training, post-training, reasoning training, GRPO/RLHF/DPO/RLVR, agent capability training, long-context extension, evaluation, safety, release, maintenance, and reproducibility. It is opinionated: in 2026, a "fully open" model should not mean only downloadable weights. It should mean a scientifically auditable system: weights, data recipes, data manifests, tokenizer, code, configs, intermediate checkpoints, training logs, evaluation harnesses, safety reports, licenses, and enough procedural detail for independent groups to challenge, reproduce, improve, or reject the results.

1. Scope And Operating Principles

1.1 What "Fully Open" Must Mean In 2026

A fully open LLM project should release more than model weights. Open weights are useful, but they do not let the research community understand why a model behaves as it does, whether a capability claim is reliable, whether a language was genuinely supported, or whether a failure comes from data, architecture, training instability, tokenizer fertility, evaluation leakage, or post-training over-optimization.

The minimum release package should contain:

- Model weights for base, instruction, reasoning, and safety variants.
- Tokenizer files, tokenizer training data description, normalization rules, and fertility analyses.
- Architecture code and exact configuration files.
- Training code, optimizer settings, parallelism settings, launcher scripts, and restart procedures.
- Data recipe, source registry, data manifests, filtering code, deduplication code, and mixture weights.
- Clear legal and licensing metadata for each data source class.
- Data-quality reports and ablation evidence.
- Intermediate checkpoints and training logs at meaningful intervals.
- Evaluation harnesses, prompts, task definitions, parse logic, raw outputs, and result summaries.
- Safety and privacy reports, including known limitations.
- Model cards and dataset cards that separate verified claims from hypotheses.
- Energy, compute, and hardware descriptions sufficient for scientific audit.

OLMo and Dolma are important precedents because they explicitly frame openness around data, code, intermediate artifacts, and logs, not only weights [@groeneveld2024olmo; @soldaini2024dolma]. FineWeb is important because it demonstrates that open data curation needs ablations and documented recipes, not only a large token count **[@penedo2024fineweb]**. OpenEuroLLM should adopt the same spirit, with an even stronger multilingual and EU-compliance emphasis.

1.2 Project Objectives

The project objective is not merely to produce a leaderboard model. A strong OpenEuroLLM-style project should optimize for five objectives at once:

1. Scientific quality: stable training, ablations, calibrated evaluation, and reproducible artifacts.
2. Multilingual usefulness: reliable coverage across EU official languages and other socially and economically relevant languages.
3. Openness: auditable training data recipes, code, logs, and model behavior.
4. Compliance and safety: EU-facing legal, privacy, copyright, and deployment risk discipline.
5. Operational usefulness: models that can be fine-tuned, served, quantized, evaluated, and integrated into real applications.

These goals interact. A model can be highly capable but not open. It can be open but too weak to matter. It can be multilingual in marketing copy while actually being evaluated mostly in English. It can be safe in English but brittle in lower-resource languages. It can score well on static benchmarks while failing as a tool-using agent. The engineering task is therefore a systems problem.

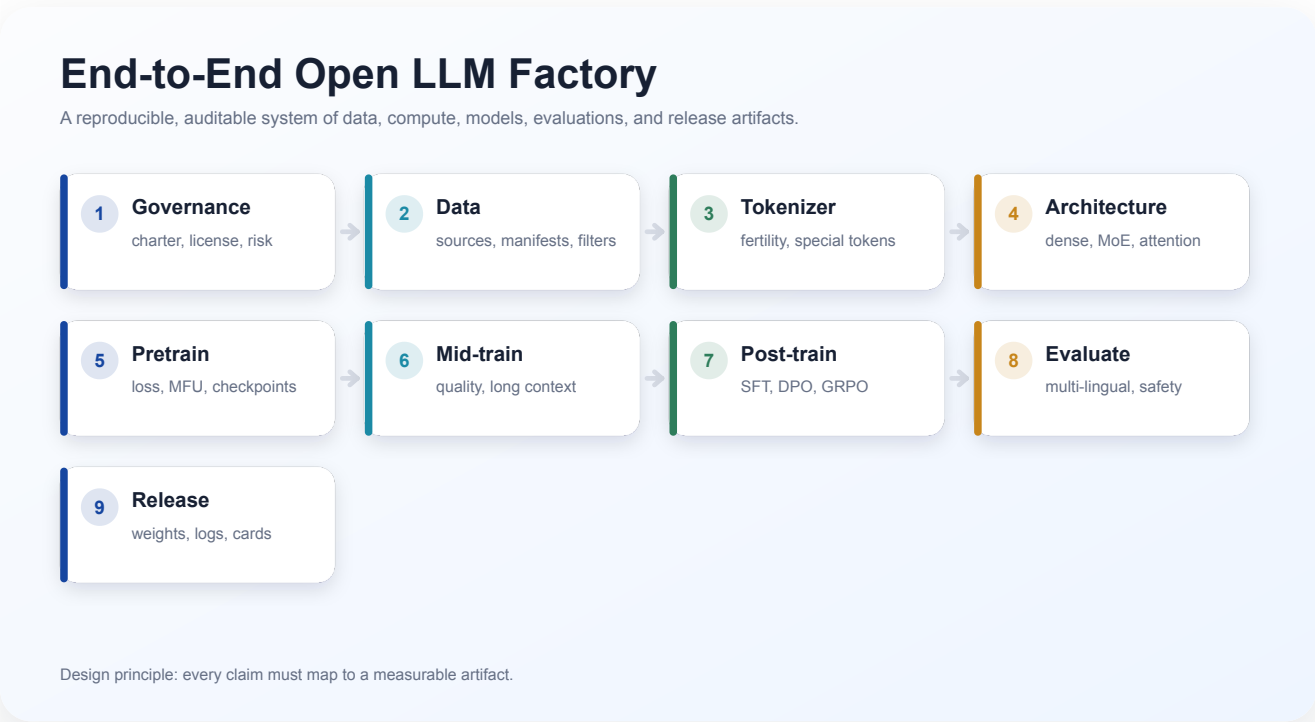
1.3 Public-Safe OpenEuroLLM Working Context

This public edition intentionally avoids non-public status details, communications, draft compute requests, and non-final workstream claims. It instead frames OpenEuroLLM as an example of the broader problem of building transparent, reproducible, multilingual, open foundation models under European governance expectations.

The actionable public lessons are:

- Treat data, model, evaluation, and release work as one auditable system.
- Keep multilingual coverage visible at every stage rather than reporting only aggregate metrics.
- Standardize training recipes across heterogeneous HPC environments.
- Separate development signals from publication-ready claims.
- Release artifacts with enough context for independent review and reuse.

2. End-To-End Lifecycle



The lifecycle of a fully open LLM can be decomposed into stages:

1. Charter, governance, and success criteria.
2. Data source registry and legal review.
3. Data extraction, normalization, and language identification.
4. Deduplication, privacy filtering, quality filtering, contamination control.
5. Dataset composition and multilingual mixture design.
6. Tokenizer design and validation.
7. Architecture selection and scaling-law experiments.
8. Infrastructure and distributed training stack.
9. Pretraining.
10. Mid-training, data curriculum, and long-context adaptation.
11. Supervised instruction tuning.
12. Preference tuning and RLHF/DPO.
13. Reasoning training with verifiable rewards, GRPO, RLVR, and distillation.
14. Agent capability training.
15. Safety tuning and refusal/calibration behavior.
16. Evaluation, red teaming, release, and monitoring.

Every stage must produce artifacts. If a stage does not produce artifacts, it is not auditable. The single most important management principle is therefore: **make the pipeline observable**. Record what data entered, what transformations were applied, what parameters were used, what was removed, what changed in validation loss, and why a decision was made.

3. Data Governance, Legal Review, And Source Registry

3.1 Source Registry

The source registry is the root of reproducibility. It should be a versioned table with one row per source collection:

- Source name and URL or storage path.
- Snapshot date and acquisition method.
- License class and terms.
- Jurisdictional considerations.
- Language/domain claims.
- Estimated documents, bytes, and tokens.
- Known risks: PII, copyright, toxicity, spam, templated content, malware, code-license issues, sensitive personal data, medical/legal content.
- Whether source text can be redistributed, redistributed as hashes/manifests, or only described as a recipe.
- Contact or owner.

For code, record license compatibility and provenance. The Stack and StarCoder work illustrate why permissive-code filtering matters for open model training [@kocetkov2022stack; @bigcode2023starcoder]. For web corpora, keep exact Common Crawl snapshots, extraction versions, filtering versions, and source-domain statistics. FineWeb's main contribution is not merely token count; it is a systematic curation recipe with ablations **[@penedo2024fineweb]**.

3.2 Legal And Privacy Review

Legal review should happen before expensive processing. The registry should classify sources into at least:

- Public-domain or permissive.
- Openly licensed with attribution or share-alike conditions.
- Web-crawled public text with uncertain copyright status.
- Contracted or partner-provided data with restrictions.
- Sensitive or high-risk data requiring exclusion or special handling.

Privacy filtering must include deterministic detectors, probabilistic models, and manual audit. At minimum:

- Email, phone, address, government ID, credit card, API keys, passwords.
- Health, legal, financial, and child-related sensitive content.
- Personal social media and forum content where identifiability is high.
- Memorization-risk analysis for rare sequences and unique documents.

The goal is not to pretend that automatic privacy filtering is perfect. The goal is to document residual risk, make conservative source choices, and provide mechanisms for removal requests and model updates.

4. Data Curation Pipeline

4.1 Extraction And Normalization

Raw data is not text. Web pages contain boilerplate, scripts, navigation, comments, tables, malformed markup, language mixing, and hidden spam. PDF and OCR content contain line breaks, hyphenation, repeated headers, and encoding errors. Code repositories contain generated files and vendored dependencies.

Recommended pipeline:

1. Extract raw text with source-specific parsers.
2. Preserve document boundaries and source metadata.
3. Normalize Unicode carefully; do not erase meaningful script distinctions.
4. Remove boilerplate, navigation, cookie notices, and repeated template text.
5. Segment documents and paragraphs.
6. Run language identification at document and segment levels.
7. Keep uncertainty scores, not only labels.
8. Store immutable raw, normalized, and filtered versions separately.

Do not use ad hoc string manipulation for major source processing. Use structured parsers where possible and keep parser versions fixed. Always sample failures visually: a 0.1% extraction bug at trillion-token scale is a very large dataset.

4.2 Language Identification

Multilingual LLMs fail quietly when language metadata is wrong. Language ID should be performed at multiple granularities:

- Document-level primary language.
- Segment-level language labels.
- Script detection.
- Confidence and entropy.
- Mixed-language markers.

For European models, distinguish languages that share scripts and vocabulary. Do not collapse regional variants too early. Use language tags such as `eng_Latn`, `deu_Latn`, or other BCP-47/Glottolog-compatible formats if the pipeline supports them.

Important metrics:

```
language_confidence = max_l p(l | document)
language_entropy = - sum_l p(l | document) log p(l | document)
mixed_document = language_entropy > tau_entropy or second_language_share > tau_share
```

Language uncertainty should affect mixture sampling and evaluation confidence. It should not simply be hidden.

4.3 Deduplication

Deduplication reduces memorization, improves data efficiency, and prevents benchmark leakage. However, excessive deduplication can harm legitimate repeated formats and minority-language data.

Use multiple deduplication levels:

- Exact document hashing.
- Near-deduplication using MinHash or SimHash.
- Line-level or paragraph-level deduplication for boilerplate.
- Cross-split deduplication between train, validation, and evaluation.
- Benchmark contamination checks.

Let D be a set of documents, $h(d)$ a locality-sensitive hash signature, and $J(d_i,d_j)$ the Jaccard similarity of shingles. Near duplicates can be defined as:

$$\mathrm{dup}(d_i,d_j)=\mathbb{1}[J(d_i,d_j) > \tau].$$

The threshold τ is not universal. Web news may need a different threshold than code, legislation, or educational material. For multilingual data, deduplication should not accidentally remove translations unless that is explicitly intended.

4.4 Quality Scoring

Quality scoring should combine interpretable filters and learned signals:

- Length and token ratio filters.
- Character distribution, punctuation, and script sanity.
- Repetition and boilerplate metrics.
- Language-ID confidence.
- Perplexity or surprisal from reference models.
- Classifiers for educational value, code quality, toxicity, spam, and domain-specific usefulness.
- LLM-as-judge sampling only for audit, not as the sole filter.

FineWeb-Edu showed that educational filtering can improve knowledge and reasoning benchmarks **[@penedo2024fineweb]**. But classifiers transfer poorly across languages and domains. For OpenEuroLLM, educational filtering must be validated per language. A classifier trained mostly on English educational text may under-score high-quality Finnish, Bulgarian, Maltese, or legal-domain text.

Recommended score:

$$q(d)=w\_\ell\, q\_\ell(d)+w_r\, q_r(d)+w_e\, q_e(d)+w_s\, q_s(d)+w_p\, q_p(d)-w_b\, b(d),$$

where q_ℓ is language confidence, q_r readability/extraction quality, q_e educational/domain value, q_s safety, q_p privacy confidence, and b is boilerplate/spam risk. The weights should be learned or tuned through ablations, not guessed once.

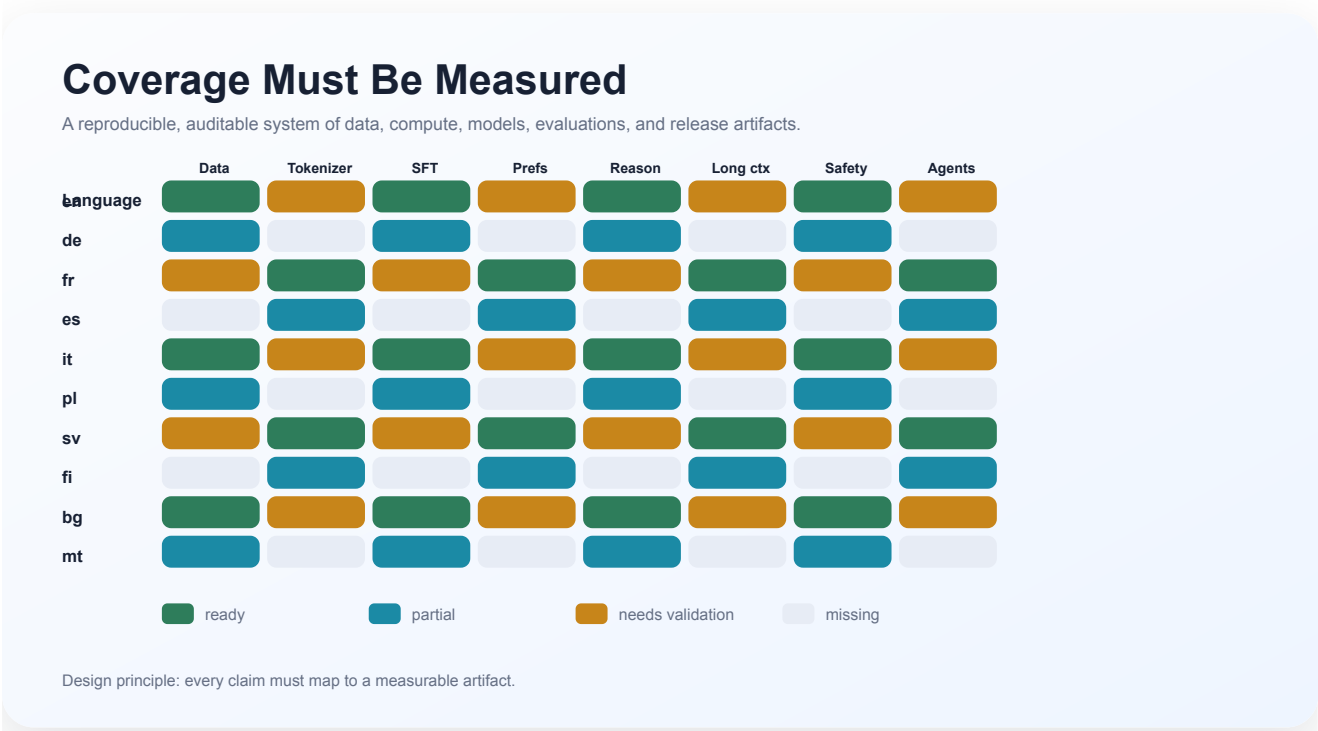
4.5 Contamination Control

Evaluation contamination is especially severe for open models because benchmark data often appears in web crawls, GitHub repositories, educational sites, and model-generated examples. The pipeline should maintain:

- Hashes and fuzzy hashes of benchmark prompts and answers.
- N-gram overlap checks.
- Semantic retrieval checks for paraphrased contamination.
- Split locks: validation and eval sets must be frozen before major training runs.
- Public reporting of contamination methodology.

Do not claim benchmark superiority without contamination analysis. For reasoning models, also check solution contamination, not only problem text.

5. Dataset Composition And Multilingual Mixture Design



5.1 Mixture As An Optimization Problem

Dataset mixture is the core of model behavior. A training mixture is not just a list of sources; it is a schedule:

$$P_t(s,l,d) = P_t(s)P_t(l|s)P_t(d|s,l),$$

where s is source, l is language, d is domain, and t is training time or curriculum phase.

The mixture should optimize:

- Validation loss across domains and languages.
- Downstream task performance.
- Robustness and safety.
- Coverage of target languages.
- Data freshness and legal reliability.
- Tokenizer efficiency and fertility.
- Avoidance of overfitting repeated high-quality subsets.

Sampling probability can be temperature-smoothed:

$$p_i = \frac{n_i^{\alpha}}{\sum_j n_j^{\alpha}},$$

where n_i is available token count for bucket i, and $\alpha \in [0,1]$ controls upsampling. $\alpha=1$ samples proportional to size; $\alpha=0$ samples uniformly across buckets. Low-resource languages usually need temperature smoothing, but aggressive upsampling risks memorization and overfitting.

5.2 Multilingual Fairness

Language coverage has at least six dimensions:

1. Pretraining token volume.
2. Tokenizer fertility.
3. Domain diversity.
4. Instruction data.
5. Preference/reasoning data.
6. Evaluation quality.

A language with many pretraining tokens but no post-training data may be fluent but bad at following instructions. A language with synthetic evaluation only may look covered but remain unvalidated. A language with high tokenizer fertility consumes more context window per word, reducing effective context.

Let F_l be tokenizer fertility for language l:

$$F_l = \frac{\text{tokens}_l}{\text{normalized words or characters}_l}.$$

Higher F_l means less text capacity per fixed token window. Long-context claims should therefore report results by language, token length, and approximate text length.

5.3 OpenEuroLLM Data Workstream

For an OpenEuroLLM-style project, the data stage should produce:

- HPLT 4.0 processing status by language and batch.
- AB sample availability and quality summary.
- Failure report and rerun plan for any unstable processing batches.
- German/Spanish release-preview status and annotation integration plan.

- BSC-edu/FineWeb-edu/OpenEuroLLM-native annotator comparison.
- Annotation capacity table by language, domain, and time.
- Locked validation split for scaling-law experiments.
- Data-card draft for every mixture version.

Any instability in large-batch processing should be treated as a visible risk register item rather than a hidden data-engineering detail.

6. Tokenizer Design

6.1 Goals

The tokenizer must support target languages, code, math, structured data, and tool-use formats without pathological fragmentation. In 2026, common choices include byte-level BPE, unigram, BBPE variants, or tokenizer families inherited from strong open models. Qwen3 reports broad multilingual support through a shared tokenizer, which reflects the practical value of a single tokenizer for model families **[@qwen3_2025]**.

Tokenizer requirements:

- Stable normalization.
- Byte fallback.
- Good fertility across all target languages.
- Robust handling of code and mathematical notation.
- Reserved tokens for chat, tools, system messages, reasoning controls, and safety metadata.
- No accidental collision between special tokens and ordinary text.
- Versioned training corpus and training script.

6.2 Fertility And Coverage Tests

For each language and domain, compute:

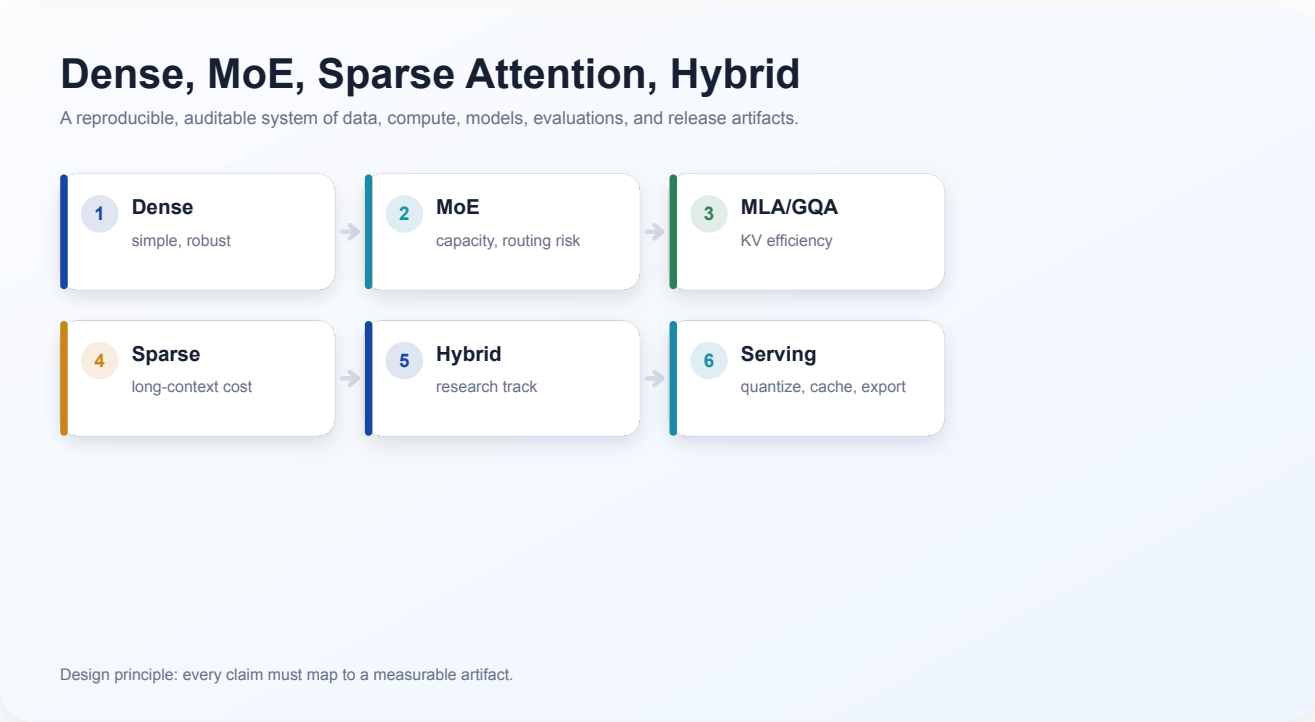
- Mean tokens per character.
- Mean tokens per word.
- Percent of single-character tokens.
- Unknown/byte fallback rate.
- Compression ratio versus UTF-8 bytes.
- Tokenization of named entities, inflections, compounds, and diacritics.
- Tokenization of code identifiers and common APIs.

Example:

```
Language: Finnish
Domain: legal text
Characters: 10,000,000
Words: 1,420,000
Tokens: 2,100,000
Tokens/word: 1.48
Bytes/token: 4.8
Fallback rate: 0.03%
```

The tokenizer should be evaluated before pretraining. Retrofitting a tokenizer after large-scale training is extremely expensive.

7. Architecture Strategy For 2026



7.1 Baseline: Dense Decoder-Only Transformer

The default baseline remains a decoder-only Transformer **[@vaswani2017attention]**. It is simple, robust, well-supported by serving stacks, and easier to debug than MoE or experimental sparse attention. A strong dense baseline is essential even if the frontier architecture becomes MoE.

Recommended dense choices:

- Pre-normalization with RMSNorm.
- SwiGLU/GeGLU-style feed-forward networks.
- RoPE or improved positional encodings.
- Grouped-query attention or multi-query attention for inference efficiency.
- QK-norm or other stabilization if validated.
- FlashAttention-compatible attention kernels [@dao2022flashattention; @flashattention2_2023].
- BF16 or FP8 training only after numerical validation.

Dense models are ideal for:

- Early scaling-law experiments.
- Reference checkpoints.

- Debugging data mixtures.
- Distillation teachers or students.
- Smaller public releases.

7.2 Mixture Of Experts

MoE is the most important architectural lever for increasing total parameter count without proportional inference cost. Switch Transformers showed the scaling promise of sparse expert routing **[@fedus2022switch]**. Mixtral demonstrated a practical open-weight sparse MoE model with strong cost/performance trade-offs **[@mixtral2024]**. DeepSeek-V3 pushed MoE further with 671B total parameters and 37B activated per token, using DeepSeekMoE, auxiliary-loss-free load balancing, MLA, FP8 training, and co-designed infrastructure **[@deepseekv3_2025]**. Qwen3 also includes dense and MoE variants **[@qwen3_2025]**.

MoE benefits:

- More total capacity for a fixed active compute budget.
- Better specialization across domains/languages if routing is stable.
- Potentially strong inference cost/performance.
- Natural family scaling: dense small models, MoE frontier models.

MoE risks:

- Routing instability.
- Expert collapse or load imbalance.
- More complex distributed training.
- Harder checkpoint portability.
- Harder inference serving and quantization.
- Multilingual routing pathologies where low-resource languages route to under-trained experts.

MoE router equations:

$$r(x)=\mathrm{softmax}(W_r\, h_x),$$

$$E(x)=\mathrm{TopK}(r(x),\, k),$$

$$y = \sum_{e} \in E(x)\; r_e(x)\; f_e(h_x).$$

Load balancing should be measured per batch, per language, per domain, and over time. A multilingual MoE must report whether experts specialize by language, domain, syntax, or

artifact. Specialization is useful only if it does not starve low-resource languages.

2026 recommendation: train a dense baseline and a MoE candidate. Use MoE for frontier scaling only after the dense pipeline is stable. Do not let MoE become the first system you debug.

7.3 Multi-Head Latent Attention And KV Efficiency

Inference cost is dominated by KV cache for long contexts. Multi-head latent attention, used in DeepSeek-V2/V3, compresses key-value representations and reduces memory pressure **[@deepseekv3_2025]**. Grouped-query attention and multi-query attention are simpler alternatives. The right choice depends on serving constraints, training code support, and quality.

Decision rule:

- If the model must serve long-context chat at scale, optimize KV cache early.
- If the model is a research baseline, prefer simpler attention.
- If adopting MLA, validate against full attention at small scale and test export/inference support before large-scale training.

7.4 Sparse Attention

Sparse attention is attractive because full self-attention has quadratic prefill cost:

$$\text{cost}_{\text{attn}} = O(n^2\, d),$$

where n is sequence length and d is hidden size. Sparse attention can reduce this toward $O(n\, k\, d)$, where $k \ll n$ is the attended subset.

2025-2026 work includes native trainable sparse attention, efficient long-sequence serving, dynamic sparse attention, and adaptive pruning [@native_sparse_attention_2025; @lserve2025; @twilight2025]. These methods are promising but operationally risky.

Sparse attention trade-offs:

- Fixed sparse masks are simple but may miss task-relevant tokens.
- Dynamic masks adapt better but are harder to train and serve.
- Inference-only sparse attention can degrade quality if the model was trained with full attention.
- Trainable sparse attention is cleaner but requires deep kernel and framework support.

2026 recommendation: keep full attention as the gold baseline. Use sparse attention for long-context frontier experiments only after measuring retrieval, lost-in-the-middle, multilingual

behavior, short-context regression, and serving compatibility. Do not claim long-context capability from nominal context length.

7.5 State-Space And Hybrid Models

Mamba-like state-space models and hybrid attention/state-space architectures remain relevant for efficient long-context modeling, but for a fully open general-purpose multilingual LLM in 2026, the safest baseline is still Transformer-first because tooling, transfer recipes, evaluation, quantization, and serving are more mature. Hybrid models are worth a research track, not the first production-scale OpenEuroLLM backbone unless the team has strong project-specific expertise.

7.6 Recommended Architecture Portfolio

Train a family:

- 0.1B-0.5B: data and tokenizer smoke tests.
- 1B-3B dense: scaling-law and mixture ablations.
- 7B-9B dense: high-quality open baseline and teacher/student anchor.
- 20B-40B dense or efficient dense: serious general model if compute allows.
- 50B-200B+ MoE total parameters: frontier candidate with lower active parameters.

Each family member should share tokenizer, data recipe lineage, evaluation harness, and model-card structure.

8. Scaling Laws And Experiment Design

8.1 Compute-Optimal Training

Scaling laws guide trade-offs between parameter count N, token count D, and compute C. A simplified training compute estimate for decoder-only Transformers is:

$$C \approx 6ND,$$

where C is FLOPs, N is non-embedding parameters, and D is training tokens. Chinchilla-style results showed that many earlier LLMs were undertrained relative to parameter count and that compute-optimal models use more tokens per parameter than older recipes **[@hoffmann2022chinchilla]**. Data-constrained scaling adds another wrinkle: if high-quality unique data is limited, repeated data and curriculum design matter **[@muennighoff2024scaling]**.

For a 9B model trained on 10T tokens:

$$C \approx 6 \times 9 \times 10^9 \times 10 \times 10^{12} = 5.4 \times 10^{23} \text{ FLOPs}.$$

At sustained 300 TFLOP/s per GPU:

$$\text{GPU seconds} = \frac{5.4 \times 10^{23}}{3 \times 10^{14}} = 1.8 \times 10^9,$$

$$\text{GPU hours} \approx 500{,}000.$$

This simplified estimate excludes overhead, failed runs, evaluation, tuning, post-training, and lower utilization. Real allocations must include overhead and reruns.

8.2 Scaling-Law Protocol

For every candidate mixture:

1. Train small models at multiple sizes.
2. Keep tokenizer and architecture fixed.
3. Train each to enough tokens to compare loss curves.

4. Evaluate on frozen validation sets by language and domain.
5. Fit loss curves:

$$L(N,D)=L_\infty + aN^{-\alpha}+bD^{-\beta}.$$

6. Compare predicted frontier performance.
7. Validate predictions with one larger run.

Do not use a single validation loss. Use a dashboard:

- General web.
- Educational.
- Code.
- Math.
- Legal/public sector.
- News.
- Low-resource languages.
- High-resource European languages.
- Instruction-like text.
- Long documents.

Scaling-law results are only useful if validation is locked, representative, and reproducible.

9. Infrastructure And Distributed Training

9.1 HPC Target Reality

OpenEuroLLM-like training happens across heterogeneous EuroHPC systems such as LUMI, Leonardo, MN5, and future project systems. Hardware differences matter:

- GPU memory: 64GB vs 80GB vs 96GB changes feasible model parallelism.
- Interconnect: affects tensor, pipeline, expert, and data parallelism.
- Filesystem performance: affects dataloader throughput and checkpointing.
- Queue policy: affects run length, failure recovery, and experiment cadence.
- Software stack: ROCm vs CUDA, compiler versions, kernel availability.

The goal is not to pretend all systems are identical. The goal is to create a portable experiment specification that compiles into system-specific launchers.

9.2 Parallelism

Large-scale training composes:

- Data parallelism (DP): replicate model, split batch.
- Tensor parallelism (TP): split matrix operations.
- Pipeline parallelism (PP): split layers across devices.
- Sequence/context parallelism (SP/CP): split sequence dimension.
- Expert parallelism (EP): distribute MoE experts.
- Fully sharded data parallelism (FSDP/ZeRO): shard parameters, gradients, and optimizer states.

Total world size:

$$W = DP \times TP \times PP \times CP \times EP.$$

The right decomposition depends on model size, sequence length, GPU memory, network, and framework support.

Dense 7B on 8x80GB GPUs may fit with TP=1, PP=1, DP=8 using activation checkpointing. A 70B dense model may need TP, PP, and FSDP. A large MoE model may need EP and careful routing all-to-all optimization.

9.3 Checkpointing And Fault Tolerance

At frontier scale, failure is normal. Checkpointing must be designed before the run:

- Save model, optimizer, scheduler, RNG, dataloader state, and consumed-token count.
- Validate restart determinism at small scale.
- Keep rolling checkpoints and milestone checkpoints.
- Store checksums and metadata.
- Monitor checkpoint time and filesystem load.
- Test partial-node failure recovery if framework supports it.

Never start a major run before a restart test has succeeded. Never assume a checkpoint is valid until it has been loaded and used to continue training.

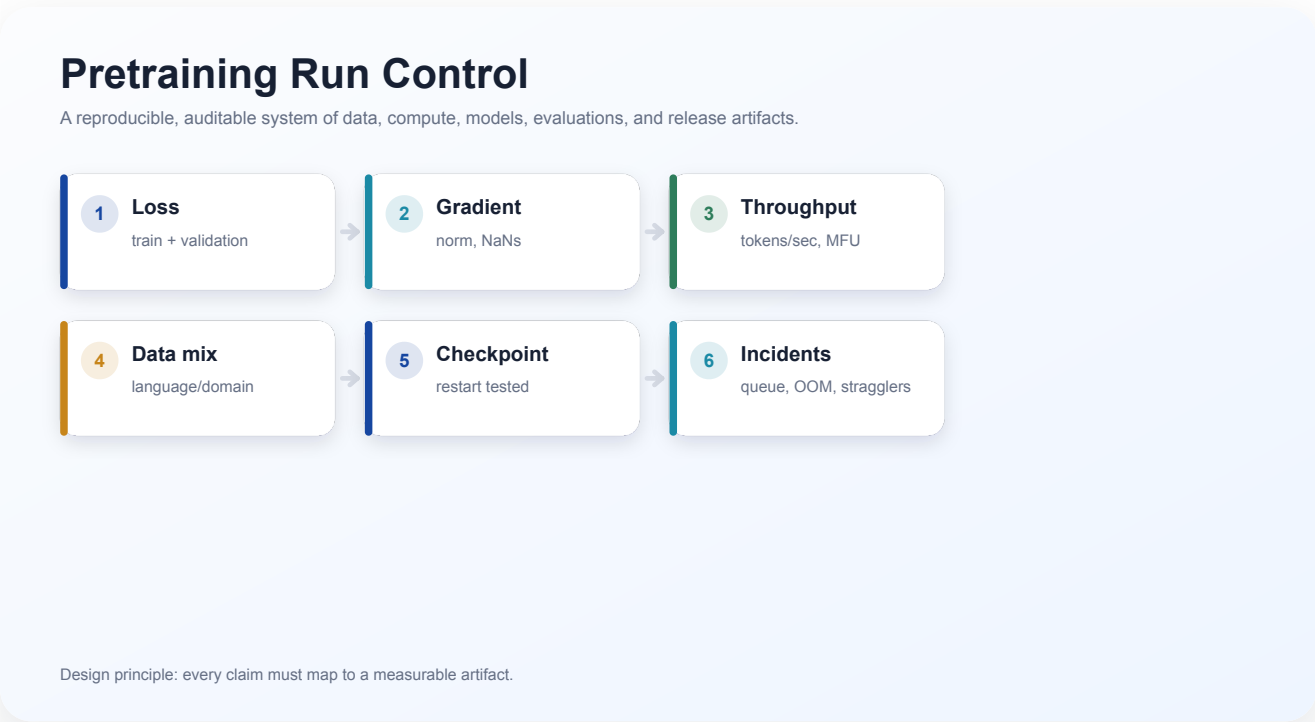
9.4 Observability

Track:

- Training loss and validation loss.
- Gradient norm.
- Learning rate.
- Tokens/sec and samples/sec.
- Model FLOP utilization (MFU).
- GPU memory and utilization.
- Dataloader time.
- Communication time.
- Straggler nodes.
- NaNs/infs.
- Expert load balance.
- Activation checkpoint overhead.
- Evaluation job latency.
- Checkpoint save/load time.

Stable training is a product feature. DeepSeek-V3 explicitly emphasizes stable training without irrecoverable loss spikes or rollbacks **[@deepseekv3_2025]**. Open projects should report stability, not only final benchmarks.

10. Pretraining



10.1 Objective

The base objective remains autoregressive next-token prediction:

$$\mathcal{L}_{LM}(\theta) = -\sum_{t=1}^{T} \log p_\theta(x_t | x_$$

Use packed sequences to reduce padding waste. But packing creates document boundary issues. For long-context training, efficient inter-document masking may matter because compute depends on actual document length statistics, not merely maximum context length. If documents are packed without masking, the model may learn cross-document artifacts. If masking is naive, it may waste attention compute. If masking is efficient and framework-supported, it can improve both quality and cost.

10.2 Hyperparameters

Key hyperparameters:

- Global batch size in tokens.
- Sequence length.
- Learning rate peak.
- Warmup tokens.
- Decay schedule.
- Weight decay.
- AdamW betas and epsilon.
- Gradient clipping.
- Dropout, often zero for large-scale pretraining.
- Initialization scale.
- Loss scaling for mixed precision.

Common schedule:

$$\eta(t) = \begin{cases} \eta_{max} \frac{t}{T_w}, & t < T_w \\ \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min}) \left(1 + \cos \frac{\pi(t - T_w)}{T - T_w}\right), & t \geq T_w \end{cases}$$

Use smaller experiments to tune learning rate and batch size. Do not copy hyperparameters blindly from a different architecture or tokenizer.

10.3 Data Curriculum

Modern strong models often use staged data curricula: broad general pretraining, then higher-quality or domain-targeted phases, then long-context or reasoning heavy phases. OLMo 2 reports a two-stage pretraining/mid-training regime and a late-stage data curriculum **[@olmo2furious2025]**. Qwen3 reports staged pretraining with general, reasoning, and long-context phases **[@qwen3_2025]**.

Recommended phases:

1. General multilingual web/books/reference/code foundation.
2. Higher-quality educational, code, math, public-sector, and multilingual curated data.
3. Long-document and long-context adaptation.
4. Optional domain-balanced cooldown to reduce over-specialization.

The curriculum should be reflected in data manifests and training logs:

```
phase: midtrain_quality_v2
start_token: 7_500_000_000_000
end_token: 9_000_000_000_000
mixture:
  hplt4_clean: 0.35
  fineweb_edu_like: 0.20
  code_permissive: 0.15
  math_science: 0.10
  eu_public_sector: 0.10
  multilingual_low_resource_upsample: 0.10
```

10.4 Validation

Evaluate during pretraining:

- Perplexity/loss by language and domain.
- Downstream zero-shot and few-shot tasks at sparse checkpoints.
- Code benchmarks.
- Math benchmarks.
- Long-context probes.
- Toxicity and refusal calibration probes.
- Memorization probes.
- Regression against previous checkpoint.

Never wait until the end to evaluate. Late discovery of data mixture failure is expensive.

11. Mid-Training And Long-Context Adaptation

11.1 Why Mid-Training Exists

Mid-training bridges raw next-token pretraining and instruction following. It can improve reasoning, code, multilingual balance, and long-context behavior without the brittleness of post-training alone.

Mid-training data can include:

- High-quality educational text.
- Textbook-like explanations.
- Code and documentation.
- Math derivations.
- Synthetic but verified reasoning traces.
- Long documents.
- Multilingual parallel and comparable corpora.
- Public-sector, legal, and administrative text.

Mid-training should still use next-token prediction unless using a specific auxiliary objective. It is not yet chat SFT.

11.2 Long Context

Long context is a system capability, not a RoPE number. Effective context length requires:

- Positional encoding support.
- Training or adaptation on long sequences.
- Attention/kernel support.
- Serving support and KV-cache capacity.
- Evaluation on retrieval, aggregation, reasoning, and lost-in-the-middle.
- Multilingual validation.

YaRN and LongRoPE are practical context-extension techniques [@peng2023yarn; @longrope2024]. The local OpenEuroLLM wiki recommends treating long context as a system-level capability and separating cheap local screening from final GPU/HPC validation.

Recommended path:

1. Baseline at original context.
2. YaRN to 32K as cheap extension baseline.
3. LongRoPE/LongRoPE2-style candidate if search and conversion overhead are manageable.
4. Evaluate short-context regression.
5. Evaluate long-context retrieval and reasoning by language.
6. Validate serving stack.

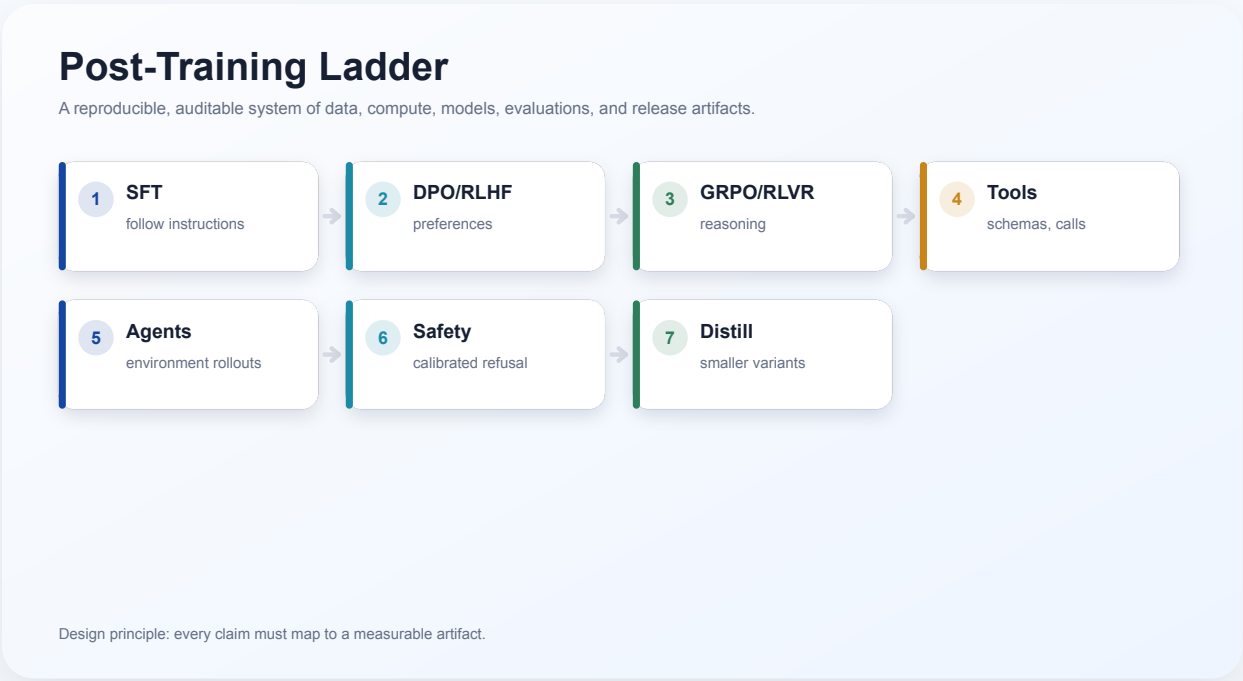
11.3 Inter-Document Masking

For packed long-context training, document boundaries matter. Suppose a packed sequence contains documents $d_1,\ldots,d_m$. Standard causal attention allows tokens in d_i to attend to previous tokens from d_i-1, which may be semantically unrelated. Inter-document masking blocks attention across document boundaries:

> $A_{ij}=1$ iff $j \le i$ and doc(i)=doc(j).

Efficient implementation is non-trivial. If efficient inter-document masking is supported, compute can depend on document length statistics such as mean μ and variance σ^2, not only maximum context. This should be tested in the actual framework.

12. Post-Training Overview



Post-training converts a base model into useful assistants, reasoners, coders, and agents. It also risks damaging the base model if done carelessly. A good post-training stack has stages:

1. Instruction supervised fine-tuning (SFT).
2. Preference optimization: RLHF, DPO, IPO/KTO-style variants.
3. Reasoning training with verifiable rewards.
4. Tool-use and agent training.
5. Safety tuning and policy calibration.
6. Distillation into smaller models.
7. Multilingual repair and regression recovery.

Llama 3, Tulu 3, DeepSeek-V3, DeepSeek-R1, and Qwen3 all show that post-training is not a minor appendix; it is central to final model behavior [@llama3herd2024; @lambert2024tulu3; @deepseekv3_2025; @deepseekr1_2025; @qwen3_2025].

13. Supervised Instruction Tuning

13.1 Data

Instruction SFT data should include:

- General chat.
- Multilingual instruction following.
- Coding.
- Math.
- Summarization.
- Information extraction.
- Structured output.
- Long-context tasks.
- Tool-call demonstrations.
- Refusal and safe-completion examples.
- Domain-specific tasks for public services and industry.

Every example should have metadata:

```
{
  "language": "deu_Latn",
"domain": "public_service",
"task": "form_explanation",
 "source": "human_written",
  "license": "cc-by-4.0",
   "safety_class": "low",
  "requires_tool": false,
   "quality_score": 0.94
}
```

13.2 Objective

For chat SFT:

$$\mathcal{L}_{SFT}(\theta) = -\sum_{(x,y)\in \mathcal{D}} \sum_t \log p_\theta(y_t|x,y_$$

Mask user tokens and train on assistant tokens unless explicitly training transcription or multi-role modeling. Preserve system-message formatting and tool-call schema exactly.

13.3 Trade-Offs

Too much SFT can:

- Reduce creativity.
- Overfit chat style.
- Harm multilingual fluency.
- Damage code or math performance.
- Increase verbosity.
- Teach brittle refusal patterns.

Use small learning rates, monitor base benchmarks, and keep multilingual regression tests.

---

14. Preference Training: RLHF, DPO, And Friends

14.1 RLHF

RLHF, popularized for instruction-following models by InstructGPT, uses human preferences to train a reward model and then optimizes the policy with PPO [@ouyang2022instructgpt; @schulman2017ppo].

Reward model:

$$\mathcal{L}_{RM} = -\log \sigma(r_\varphi(x,y_w)-r_\varphi(x,y_l)),$$

where y_w is preferred and y_l is rejected.

Policy objective with KL control:

$$\max_\theta \mathbb{E}_{y\sim \pi_\theta}\left[r_\varphi(x,y) - \beta D_{KL}(\pi_\theta(\cdot|x)\|\pi_{ref}(\cdot|x)) \right].$$

RLHF strengths:

- Flexible preference modeling.
- Can optimize nuanced human judgments.
- Useful for style, helpfulness, harmlessness, and interaction quality.

RLHF risks:

- Reward hacking.
- Expensive human labeling.
- Reward model bias.
- Instability.
- Multilingual preference gaps.
- Over-optimization and loss of diversity.

14.2 DPO

Direct Preference Optimization removes the explicit reward model and optimizes preferences directly **[@rafailov2023dpo]**:

$$\mathcal{L}_{DPO}(\pi_\theta;\pi_{ref}) = -\mathbb{E}_{(x,y_w,y_l)} \log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)} \right).$$

DPO strengths:

- Simpler than PPO.
- Stable.
- Good for open post-training pipelines.
- Easy to reproduce.

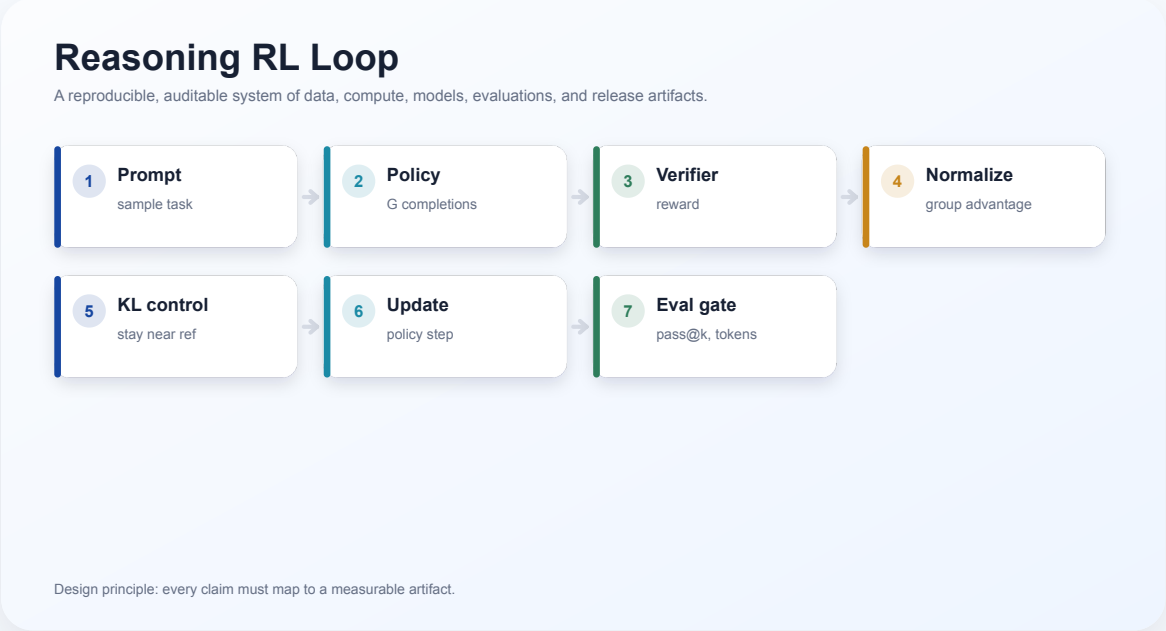
DPO risks:

- Depends heavily on preference data quality.
- Less flexible for multi-step environment rewards.
- Can overfit preference style.

2026 recommendation: use SFT + DPO as the default reliable open pipeline. Use RLHF/PPO or GRPO/RLVR when the reward is verifiable, environment-based, or

when the project has the infrastructure to monitor policy optimization carefully.

15. Reasoning Training



15.1 Why Reasoning Needs Its Own Stage

Reasoning training is not just "more chat data". It needs tasks with delayed credit assignment, verifiable answers, chain-of-thought or latent reasoning behavior, and careful evaluation. DeepSeekMath introduced GRPO for mathematical reasoning in open models **[@deepseekmath2024]**. DeepSeek-R1 demonstrated the importance of reinforcement learning for reasoning behavior, including long-form thinking and self-correction **[@deepseekr1_2025]**. Qwen3's thinking/non-thinking modes indicate a 2026 trend: models should know when to reason deeply and when to answer efficiently **[@qwen3_2025]**.

15.2 Reasoning Data Types

- Math problems with exact answers.
- Code problems with unit tests.
- Logic puzzles.
- Scientific QA with derivable answers.
- Formal proof steps.
- Data-analysis tasks with executable checks.

- Multi-hop retrieval with cited evidence.
- Tool-augmented tasks with environment verification.

Each example should define whether the reward is:

- Exact match.
- Unit test pass.
- Symbolic equivalence.
- Verifier-model judgment.
- Human preference.
- Hybrid.

Prefer verifiable rewards where possible.

15.3 GRPO

Group Relative Policy Optimization avoids a separate value model by sampling a group of completions for each prompt and normalizing rewards within the group. For prompt x, sample completions $y_1,\ldots,y_G$, compute rewards r_i, then:

$$\bar{r}=\frac{1}{G}\sum_{i=1}^{G} r_i,$$
$$\sigma_r = \sqrt{(\frac{1}{G}\sum_i (r_i-\bar{r})^{2}+\varepsilon)},$$

$$A_i=\frac{r_i-\bar{r}}{\sigma_r}.$$

A simplified GRPO-style objective:

$$\mathcal{L}_{GRPO}(\theta)= -\frac{1}{G}\sum_{i=1}^{G} \min(\ \rho_i A_i,\ clip(\rho_i,1-\varepsilon,1+\varepsilon)A_i\ ) \setminus + \beta\ D_{KL}(\pi\_\theta||\pi_{ref}),$$

where

$$\rho_i = \frac{\pi\_\theta(y_i|x)}{\pi_{\theta_{old}}(y_i|x)}.$$

GRPO strengths:

- No value model.
- Good for verifiable reasoning tasks.
- Efficient group-relative signal.
- Works naturally with multiple sampled solutions.

GRPO risks:

- Group size affects variance and cost.
- Reward hacking.
- Format overfitting.
- Long reasoning traces can become verbose or self-indulgent.
- Multilingual reasoning may lag if rewards are English-centric.

15.4 RLVR

Reinforcement learning with verifiable rewards (RLVR) should be the default for math/code/science reasoning where possible. The reward function should be transparent:

```
def reward(problem, answer):
    parsed = parse_final_answer(answer)
    if not parsed.valid:
        return -0.2
    if equivalent(parsed.value, problem.gold):
        return 1.0
    return 0.0
```

For code:

```
def reward(problem, completion):
    program = extract_code(completion)
    result = run_tests_in_sandbox(program, problem.tests)
    return result.passed / result.total
```

Reward design must penalize invalid formatting, unsafe tool calls, and non-termination. But do not over-penalize alternative reasoning paths if the answer is correct.

15.5 Thinking Budgets

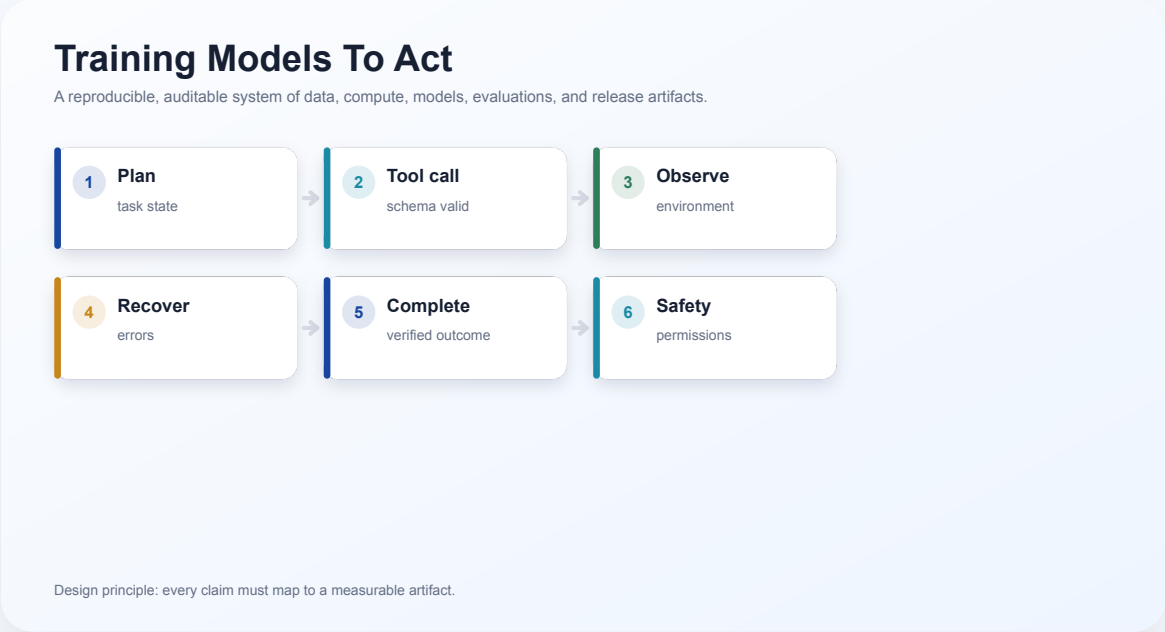
Qwen3-style thinking/non-thinking modes suggest a practical interface: users or systems can allocate a reasoning budget **[@qwen3_2025]**. Train the model with

explicit control tokens or system instructions:

```
<mode>fast</mode> Answer directly.
<mode>think</mode> Use a hidden scratchpad, then provide the concise answer.
<budget>2048</budget>
```

For open models, be careful with public chain-of-thought. A useful compromise is hidden reasoning during training and concise rationales at inference. If the project releases reasoning traces, filter for privacy, hallucinated citations, and unsafe procedural content.

16. Agent Capability Training



16.1 What Agent Capability Means

Agent capability means the model can plan, use tools, observe results, recover from errors, and complete tasks across multiple steps. It is not the same as chat helpfulness.

Agent tasks include:

- Search with citation.
- Code editing.
- Running tests.
- Browser navigation.

- Spreadsheet/document manipulation.
- API use.
- Database querying.
- Multi-step research.
- Workflow automation.

ReAct and Toolformer are early foundations for reasoning-plus-acting and tool use [@yao2022react; @schick2023toolformer]. Llama 3 explicitly includes tool usage as a native capability target **[@llama3herd2024]**. In 2026, agent training should be a first-class post-training stage.

16.2 Agent Data

Agent training data:

- Human tool-use traces.
- Synthetic tool-use tasks with verified outcomes.
- Code repair trajectories.
- Browser tasks.
- Document editing tasks.
- API schema following.
- Failure recovery examples.
- Permission-denial examples.
- Cost-aware examples.

Trajectory format:

```
{
"task": "Fix the failing unit test and summarize the patch.",
"messages": [
{"role": "user", "content": "..."},
{"role": "assistant", "content": "I will inspect the failure."},
{"role": "assistant", "tool_call": {"name": "run_tests", "args": {}}},
{"role": "tool", "content": "1 failing test..."},
{"role": "assistant", "tool_call": {"name": "edit_file", "args": {...}}},
{"role": "assistant", "content": "Fixed..."}
],
"outcome": "tests_pass",
"reward": 1.0
}
```

16.3 Agent Evaluation

Metrics:

- Task completion.
- Tool-call validity.
- Number of unnecessary tool calls.
- Recovery from tool errors.
- Permission safety.
- Sandbox compliance.
- Citation correctness.
- Patch correctness.
- Latency and cost.
- Human preference.

Agent models must learn to ask for permission when needed, avoid destructive actions, and keep working through recoverable failures.

16.4 Training Methods

Stages:

1. Tool schema SFT.
2. Demonstration trajectory SFT.
3. Environment rollouts with reward.
4. Preference tuning on trajectory quality.
5. Safety tuning for permissions and irreversible actions.
6. Regression tests on non-agent chat tasks.

Use RL for environments with objective success signals. Use preference training for style, helpfulness, and judgment. Do not use free-form model-judge rewards alone for high-stakes agent behavior.

17. Safety, Compliance, And Alignment

17.1 Safety Is Multilingual

Safety behavior must be evaluated across languages. A refusal policy that works in English may fail in Polish, Swedish, Maltese, Bulgarian, or code-switched prompts.

Safety data must include:

- Direct harmful requests.
- Translated harmful requests.
- Code-switched harmful requests.
- Indirect prompt injection.
- Tool-use abuse.
- Public-sector sensitive scenarios.
- Medical/legal/financial boundary cases.
- Privacy extraction attempts.

17.2 Refusal Calibration

Good safety is not maximum refusal. It is calibrated helpfulness:

- Answer benign questions.
- Redirect dangerous instructions.
- Provide safe alternatives.
- Avoid moralizing verbosity.
- Preserve multilingual quality.

Track:

- False refusal rate.
- False compliance rate.
- Helpfulness after refusal.
- Language-specific refusal disparities.
- Jailbreak robustness.

17.3 Model And Dataset Cards

Every release should include:

- Intended uses.
- Out-of-scope uses.
- Data summary.
- Training stages.
- Evaluation results.
- Safety results.
- Known limitations.
- Language coverage.
- License and compliance notes.
- Contact and removal process.

Model cards should not overclaim. If a language has synthetic evaluation only, say so.

---

18. Evaluation

18.1 Evaluation Taxonomy

Evaluate:

- Base language modeling.
- Knowledge.
- Reasoning.
- Math.
- Code.
- Multilingual tasks.
- Translation and cross-lingual transfer.
- Long context.
- Instruction following.
- Tool use.
- Agent tasks.
- Safety.
- Bias and representational harms.
- Robustness.
- Memorization and privacy.

18.2 Multilingual Evaluation

For OpenEuroLLM-style multilingual evaluation, synthetic fallback resources should require native-speaker validation before publishable claims. Make evaluation status explicit:

- Native validated.
- Professional translation.
- Community reviewed.
- Synthetic fallback.
- Missing.

Report parse failure rate:

$$\text{parse failure rate} = \frac{\#\text{unparseable outputs}}{\#\text{total outputs}}.$$

This matters because models often "know" the answer but fail the format, or produce unparseable multilingual output.

18.3 Long-Context Evaluation

Long-context benchmarks:

- Single needle retrieval.
- Multi-needle retrieval.
- Lost-in-the-middle curves.
- Variable tracking.
- Long-document QA.
- Cross-lingual context/instruction.
- Long-context perplexity.
- Multi-document synthesis with citations.

Report performance by context length and position:

$$\text{accuracy}(l,p,n)$$

where l is language, p is evidence position, and n is context length.

18.4 Reasoning Evaluation

Reasoning evaluation should include pass@k:

$$\text{pass@k} = 1 - \frac{C(n-c, k)}{C(n, k)},$$

where n completions are sampled and c are correct.

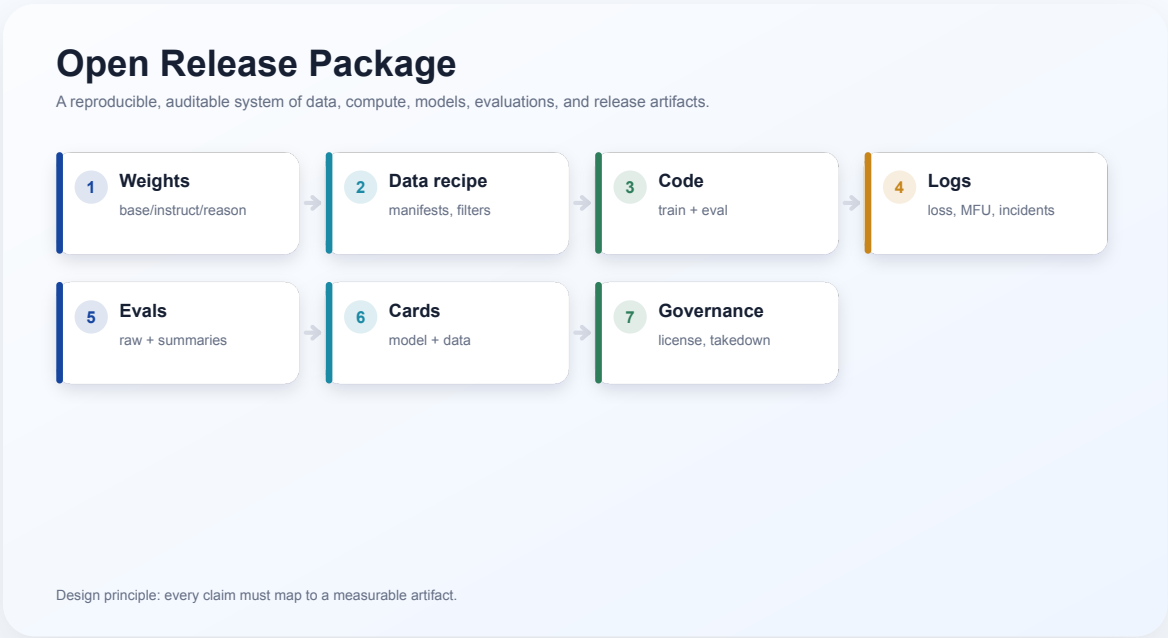
But pass@k can hide verbosity, invalid formatting, and compute cost. Report:

- pass@1, pass@8, pass@32.
- Tokens per solution.
- Verifier pass rate.
- Invalid output rate.
- Self-correction success.
- Language-specific reasoning performance.

18.5 Agent Evaluation

Agent evals should run in sandboxes and measure actual outcomes. For coding, use tests. For browser tasks, use DOM state. For research tasks, use citation precision and answer faithfulness. For document tasks, render outputs and verify layout.

19. Release Engineering



19.1 Release Tiers

Release:

- Base model.
- Instruct model.
- Reasoning model.
- Agent model if trained.
- Safety classifier or guard model.
- Smaller distilled variants.
- Quantized variants.

Each release should have a reproducible build path.

19.2 Artifact Manifest

Example:

```
model_name: openeurollm-9b-base-v1
architecture: dense_decoder_transformer
tokenizer: openeurollm-tokenizer-v1
training_tokens: 10_000_000_000_000
context_length: 8192
long_context_variant: yarn-32768
data_recipe: mixture-v4.2
code_commit: abc123
checkpoint: step_1000000
license: apache-2.0-compatible-weights
eval_report: evals/openeurollm-9b-base-v1.json
model_card: README.md
```

19.3 Intermediate Checkpoints

Intermediate checkpoints are scientifically valuable. Release them at:

- Early stable checkpoint.
- 25%, 50%, 75%, 100% of training.
- Before and after mid-training.
- Before and after post-training.

This supports mechanistic interpretability, data studies, and reproducibility.

---

20. OpenEuroLLM-Specific Technical Roadmap

20.1 Immediate

- Stabilize large-scale corpus processing and document rerun plans.
- Publish a data status dashboard by language and batch when the information is approved for public release.
- Lock validation split for scaling laws.
- Standardize training configs across LUMI, Leonardo, MN5, and the fourth system.
- Convert ad hoc Slurm scripts into `oellm-autoexp` recipes where possible.
- Define minimum multilingual evaluation matrix in `oellm-cli`.
- Create model-card template with required metadata.

20.2 Near-Term

- Run dense small-model ablations for data mixture.
- Compare BSC-edu, FineWeb-edu-style, and OpenEuroLLM-native annotators.
- Validate tokenizer fertility across target languages.
- Train 0.4B-2B reference models for mixture and infrastructure tests.
- Create long-context screening harness.
- Build post-training data registry.

20.3 Medium-Term

- Train 7B-9B dense base model.
- Train instruction and reasoning variants.
- Run GRPO/RLVR experiments on math/code/verifiable multilingual tasks.
- Build tool-use and agent SFT data.
- Evaluate MoE candidate with expert-load diagnostics by language.
- Prepare compute-access fallback plans for partial allocation.

20.4 Long-Term

- Train frontier dense or MoE model.
- Release fully open artifacts.
- Maintain evaluation leaderboard and issue tracker.
- Support community fine-tuning.
- Maintain removal and correction processes.
- Publish technical report with training logs, ablations, and limitations.

21. Trade-Off Summary

Decision	Best 2026 Default	Upside	Risk
Dense vs MoE	Dense baseline plus MoE candidate	Reliable baseline and scalable frontier	MoE complexity
Full vs sparse attention	Full baseline; sparse as long-context track	Quality and debuggability	Sparse serving mismatch
SFT vs DPO/RLHF	SFT + DPO default	Stable open post-training	May underperform RL on reasoning
GRPO/RLVR	Use for verifiable reasoning	Strong reasoning gains	Reward hacking
Agent training	Separate tool/agent stage	Real task capability	Safety and eval complexity
Long context	System-level validation	Real long-doc utility	Nominal context overclaims
Data openness	Manifests + recipes + filters	Auditability	Legal complexity

22. The "World-Class" Checklist

A world-class fully open LLM project in 2026 should be able to answer:

- What exact data was used?
- What was removed and why?
- Which languages are genuinely covered?
- How was tokenizer quality measured?
- What architecture alternatives were tested?
- What scaling-law evidence justified the final run?
- What was the sustained MFU?
- How many restarts happened?
- What were the largest training incidents?
- How did validation loss evolve by language and domain?
- What post-training data changed behavior?
- Which rewards were verifiable?
- Did reasoning training harm normal chat?
- Did agent training create unsafe tool behavior?

- Which benchmarks may be contaminated?
- Which claims are native-speaker validated?
- What are the known limitations?
- Can another lab reproduce the recipe?

If the answer to any of these is "we do not know", the release can still be valuable, but the uncertainty must be disclosed.

---

23. Conclusion

Training a fully open LLM in 2026 is not a single training run. It is a multi-year scientific infrastructure project. The model is only one artifact. The real deliverable is an open system of evidence: data lineage, architecture decisions, training stability, post-training methodology, evaluation discipline, safety analysis, and release reproducibility.

For OpenEuroLLM, the opportunity is unusually important. The project can define what openness means for European multilingual AI: not merely open weights, but open science, open evaluation, open recipes, and honest multilingual coverage. The strongest path is pragmatic: build a stable dense baseline, run disciplined data and scaling ablations, standardize HPC recipes, evaluate multilingual coverage rigorously, add post-training in carefully measured stages, use GRPO/RLVR where rewards are verifiable, train agent capabilities explicitly, and release the artifacts needed for others to inspect and improve the work.

The central rule is simple: if a claim matters, make it measurable; if a result matters, make it reproducible; if a risk matters, make it visible.

References

Use `references.bib` with Pandoc, Quarto, or another citation-aware renderer to format the bibliography.

Linked References

Each citation below includes a URL to the paper, technical report, official project page, or local source descriptor used by the manuscript.

1. `@openeurollm_official_2026` **OpenEuroLLM: Transparent AI for Europe**. OpenEuroLLM Consortium (2026). *Official project website, accessed 2026-05-07*.
   https://www.openeurollm.eu/
2. `@ai_sweden_openeurollm_2026` **OpenEuroLLM**. AI Sweden (2026). *AI Sweden project page, accessed 2026-05-07*.
   https://www.ai.se/en/project/openeurollm
3. `@vaswani2017attention` **Attention Is All You Need**. Vaswani Ashish et al. (2017). *Advances in Neural Information Processing Systems*.
   https://arxiv.org/abs/1706.03762
4. `@kaplan2020scaling` **Scaling Laws for Neural Language Models**. Kaplan Jared et al. (2020).
   https://arxiv.org/abs/2001.08361
5. `@hoffmann2022chinchilla` **Training Compute-Optimal Large Language Models**. Hoffmann Jordan et al. (2022).
   https://arxiv.org/abs/2203.15556
6. `@touvron2023llama` **LLaMA: Open and Efficient Foundation Language Models**. Touvron Hugo et al. (2023).
   https://arxiv.org/abs/2302.13971
7. `@llama3herd2024` **The Llama 3 Herd of Models**. Dubey Abhimanyu et al. (2024).
   https://arxiv.org/abs/2407.21783
8. `@groeneveld2024olmo` **OLMo: Accelerating the Science of Language Models**. Groeneveld Dirk et al. (2024).
   https://arxiv.org/abs/2402.00838
9. `@olmo2furious2025` **2 OLMo 2 Furious**. OLMo Team (2025).
   https://arxiv.org/abs/2501.00656

10. `@soldaini2024dolma` **Dolma: an Open Corpus of Three Trillion Tokens for Language Model Pretraining Research**. Soldaini Luca et al. (2024).
https://arxiv.org/abs/2402.00159

11. `@penedo2024fineweb` **The FineWeb**. Penedo Guilherme et al. (2024). *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.
https://proceedings.neurips.cc/paper_files/paper/2024/hash/370df50ccfdf8bde18f8f9c2d9151bda-Abstract-Datasets_and_Benchmarks_Track.html
DOI: 10.52202/079017-0970

12. `@li2024dclm` **DataComp-LM: In search of the next generation of training sets for language models**. Li Jeffrey et al. (2024).
https://arxiv.org/abs/2406.11794

13. `@fedus2022switch` **Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity**. Fedus William et al. (2022). *Journal of Machine Learning Research*.
https://arxiv.org/abs/2101.03961

14. `@mixtral2024` **Mixtral of Experts**. Mistral AI (2024).
https://arxiv.org/abs/2401.04088

15. `@deepseekv3_2025` **DeepSeek-V3 Technical Report**. DeepSeek-AI (2025).
https://arxiv.org/abs/2412.19437

16. `@qwen3_2025` **Qwen3 Technical Report**. Yang An et al. (2025).
https://arxiv.org/abs/2505.09388

17. `@deepseekmath2024` **DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models**. Shao Zhihong et al. (2024).
https://arxiv.org/abs/2402.03300

18. `@deepseekr1_2025` **DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning**. DeepSeek-AI (2025).
https://arxiv.org/abs/2501.12948

19. `@ouyang2022instructgpt` **Training language models to follow instructions with human feedback**. Ouyang Long et al. (2022).
https://arxiv.org/abs/2203.02155

20. `@rafailov2023dpo` **Direct Preference Optimization: Your Language Model is Secretly a Reward Model**. Rafailov Rafael et al. (2023).
https://arxiv.org/abs/2305.18290

21. `@lambert2024tulu3` **Tulu 3: Pushing Frontiers in Open Language Model Post-Training**. Lambert Nathan et al. (2024).
https://arxiv.org/abs/2411.15124

22. `@schulman2017ppo` **Proximal Policy Optimization Algorithms**. Schulman John et al. (2017).
https://arxiv.org/abs/1707.06347

23. `@yao2022react` **ReAct: Synergizing Reasoning and Acting in Language Models**. Yao Shunyu et al. (2022).
https://arxiv.org/abs/2210.03629

24. `@schick2023toolformer` **Toolformer: Language Models Can Teach Themselves to Use Tools**. Schick Timo et al. (2023).
https://arxiv.org/abs/2302.04761

25. `@longrope2024` **LongRoPE: Extending LLM Context Window Beyond 2 Million Tokens**. Ding Yuhui et al. (2024).
https://arxiv.org/abs/2402.13753

26. `@peng2023yarn` **YaRN: Efficient Context Window Extension of Large Language Models**. Peng Bowen et al. (2023).
https://arxiv.org/abs/2309.00071

27. `@dao2022flashattention` **FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness**. Dao Tri et al. (2022).
https://arxiv.org/abs/2205.14135

28. `@flashattention2_2023` **FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning**. Dao Tri (2023).
https://arxiv.org/abs/2307.08691

29. `@native_sparse_attention_2025` **Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention**. Yuan Jingyang et al. (2025).
https://arxiv.org/abs/2502.11089

30. `@lserve2025` **LServe: Efficient Long-sequence LLM Serving with Unified Sparse Attention**. Yang Shang et al. (2025).
https://arxiv.org/abs/2502.14866

31. `@twilight2025` **Twilight: Adaptive Attention Sparsity with Hierarchical Top-p Pruning**. Lin Chaofan et al. (2025). *Advances in Neural Information Processing Systems*.
https://research.nvidia.com/labs/eai/publication/twilight/

32. `@muennighoff2024scaling` **Scaling Data-Constrained Language Models**. Muennighoff Niklas et al. (2024).
https://arxiv.org/abs/2305.16264

33. `@bigcode2023starcoder` **StarCoder: May the source be with you!**. Li Raymond et al. (2023).
https://arxiv.org/abs/2305.06161

34. `@kocetkov2022stack` **The Stack: 3 TB of permissively licensed source code**. Kocetkov Denis et al. (2022).
https://arxiv.org/abs/2211.15533